



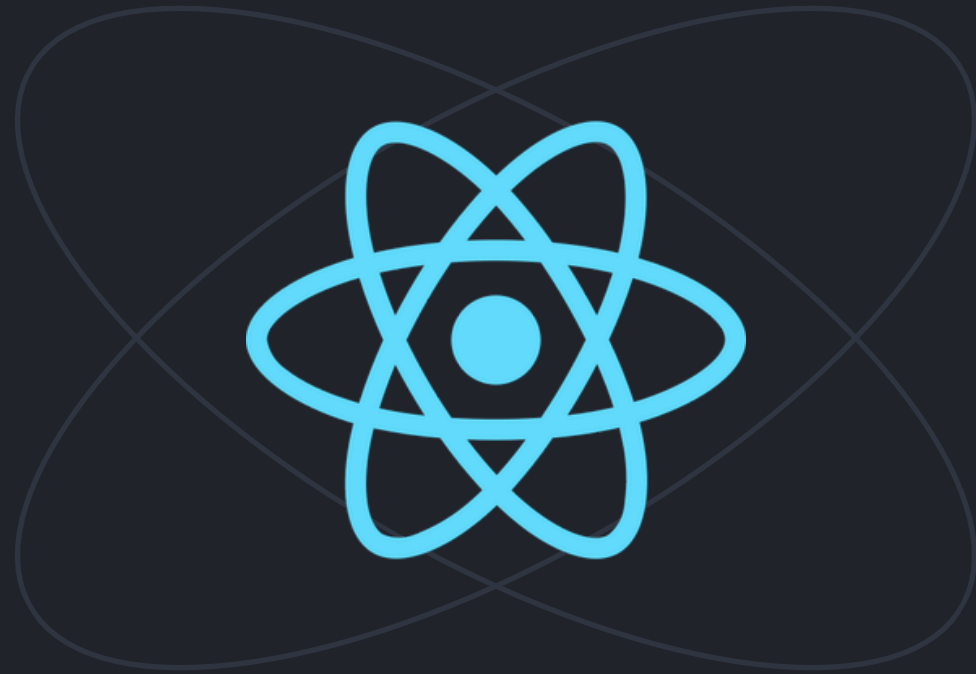
SE 321

Lecture 6: Intro to **React.js**

Mahbubur Rahman

Lecturer

Department of Software Engineering
Daffodil International University



Introduction to React.js

Building interactive websites, one component at a time

Components

State

Events

Forms

Lists

Learning roadmap

02

1

Components

Break the page into reusable parts

2

State

Remember what changes on the screen

3

Events

Respond to click, type, submit

4

Forms

Connect inputs to state

5

Lists

Display many items from an array

Core idea

React keeps your user interface in sync with data. When state changes, React updates the visible UI for you.

The Problem: Modern Websites Are Complicated



The page changes constantly

Likes, comments, carts, notifications — the screen must update every second, without reloading.



Manual DOM updates get messy

With plain JS you call `getElementById(...)` everywhere and update each element by hand. Easy to miss one.



Code repeats itself

The same card, button, or menu is copy-pasted across pages. Fixing a bug means fixing it 20 times.



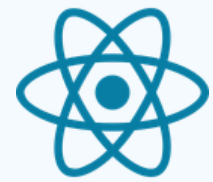
Think of one Facebook 'Like'

Tapping **one heart** must instantly update: the like counter, the button colour, your activity log, and the post owner's notifications — all **without refreshing the page**.

React's promise: you describe what the UI should look like — React keeps the page updated for you.

So, What Exactly Is React?

A JavaScript library for building user interfaces



React is a JavaScript library created at Facebook (2013) for building UIs out of small, reusable pieces called **components**. It is not a new language — it is JavaScript with superpowers.



Component-based

Build small pieces (a button, a card), then snap them together into full pages.



Declarative

You declare the final look; React figures out which parts of the page to change.



Fast updates

React updates only what changed — not the whole page — so apps feel instant.

REAL WORLD: Netflix, Instagram, Airbnb, WhatsApp Web and Facebook itself are all built with React.

Components: The LEGO Bricks of a Page

Every screen is just components inside components

A component

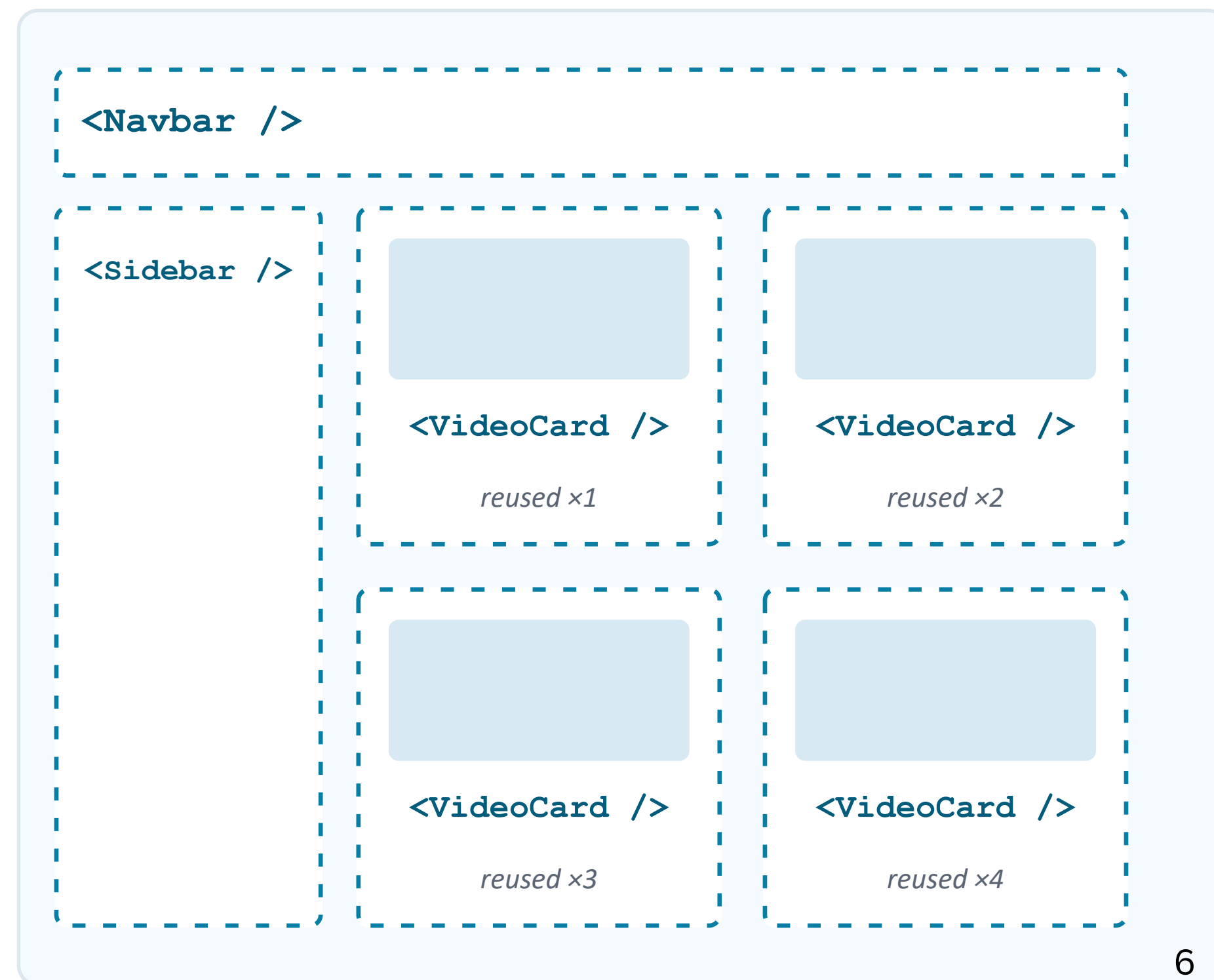
is a small, self-contained piece of UI — its own look and its own logic — that you can reuse anywhere, as many times as you like.



Write once, use everywhere

One `<VideoCard />` component renders every video on the page. Fix it once, it is fixed everywhere.

REAL WORLD: Open YouTube: the navbar, the sidebar and every video card are separate React components.



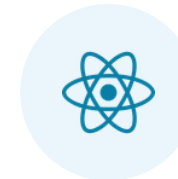
Your First Component

It's just a JavaScript function that returns HTML-like code

```
● ● ● Welcome.jsx

// A component = a JS function
// Name MUST start with a capital letter
function Welcome() {
  return (
    <div>
      <h1>Hello, class!</h1>
      <p>My first component</p>
    </div>
  );
}

// Use it like a custom HTML tag:
<Welcome />
```



This is JSX

HTML written inside JavaScript. The browser doesn't read it directly — React translates it for you.



Return one parent tag

A component must return a single wrapping element, like the `<div>` here.



Reuse it anywhere

`<Welcome />` can now be dropped into any page — as often as you like.

State: A Component's Memory

Data that changes while the app is running

State is the data a component remembers between clicks and keystrokes: a like count, the text in a search box, whether a menu is open.

The golden rule: when state changes, React automatically re-draws the component. You never touch the DOM yourself.

REAL WORLD: Instagram's like counter and Amazon's cart badge are state. Tap once — the number changes instantly, no page reload.

The state cycle



1 · User clicks the ♥ button



2 · Code updates state: likes + 1



3 · React re-renders the component



4 · Screen now shows the new count

The useState Hook

How a component gets its memory

LikeButton.jsx

```
import { useState } from 'react';

function LikeButton() {
  // [value, updater function]
  const [likes, setLikes] = useState(0);

  return (
    <button onClick={() => setLikes(likes +
1)} >
      ❤️ {likes} likes
    </button>
  );
}
```

useState(0)

Creates state with a starting value of 0.

likes

The current value — read it anywhere in the component.

setLikes(...)

The ONLY correct way to change it. Never write likes = 5.

{likes} in JSX

Curly braces drop any JS value into your HTML.

Handling Events: Making It Interactive

Reacting to clicks, typing, hovering — like HTML, but cleaner



camelCase names

HTML: `onclick="..."` → React: `onClick=`
`{...}`. Same idea for `onChange`, `onSubmit`,
`onMouseOver`.



Pass a function, not a string

You hand React a real JS function to call
— no strings of code.

REAL WORLD: Netflix's Play button, YouTube's
subscribe button, dark-mode toggles — every tap on
the web is an event handler.

```
DarkModeToggle.jsx

function DarkModeToggle() {
  const [dark, setDark] =
    useState(false);

  // the event handler function
  function handleClick() {
    setDark(!dark); // flip it!
  }

  return (
    <button onClick={handleClick}>
      {dark ? '🌙 Dark' : '☀️ Light'}
    </button>
  );
}
```

Conditional Rendering: Show or Hide

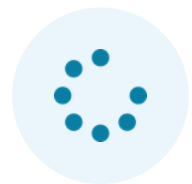
Different UI for different situations — using plain JS logic

There is no special syntax. You use the JavaScript you already know — if, the ternary `? :`, and `&&` — right inside JSX.



**cond ? <A/> : **

Show A when true, otherwise B. Perfect for login vs. logout buttons.



cond && <A/>

Show A only when true, otherwise nothing. Perfect for loading spinners and error messages.

```
Profile.jsx
function Profile({ isLoggedIn }) {
  return (
    <nav>
      {isLoggedIn
        ? <button>Log out</button>
        : <button>Sign in</button>}

      {isLoggedIn && <p>Welcome
back!</p>}
    </nav> ); }

```

REAL WORLD: Facebook shows 'Log in' or your profile picture — same page, different state.

Controlled Forms: React Owns the Input

State becomes the single source of truth for what's typed

In plain HTML, the browser remembers what you typed. In **React**, we flip that: **state holds the text, and the input just displays it**. That's a controlled component.

Why bother? Because once the text lives in state, you can validate it, filter with it, or send it — instantly, on every keystroke.

REAL WORLD: Google's search box shows suggestions as you type — every keystroke updates state, which updates the suggestion list.

The controlled-input loop



You type a letter



onChange fires with the new text



setText(...) saves it in state



value={text} shows it in the box

A Controlled Form in Code

value + onChange + onSubmit — the three-piece pattern

SignupForm.jsx

```
function SignupForm() {
  const [email, setEmail] = useState('');

  function handleSubmit(e) {
    e.preventDefault(); // stop page reload
    alert('Welcome ' + email);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={email}           // state → box
        onChange={e => setEmail(e.target.value)}
      />
      <button>Sign up</button>
    </form>
  );
}
```

value={email}

The box always shows the state.

onChange

Every keystroke updates the state.

e.preventDefault()

Stops the old-school full-page reload on submit.

Memorise this trio — it powers every React form.

Rendering Lists: From Data to UI

One array + one component = an entire feed

Real apps don't hard-code 50 cards. They keep the data in an array and use JavaScript's `.map()` to turn each item into a component.



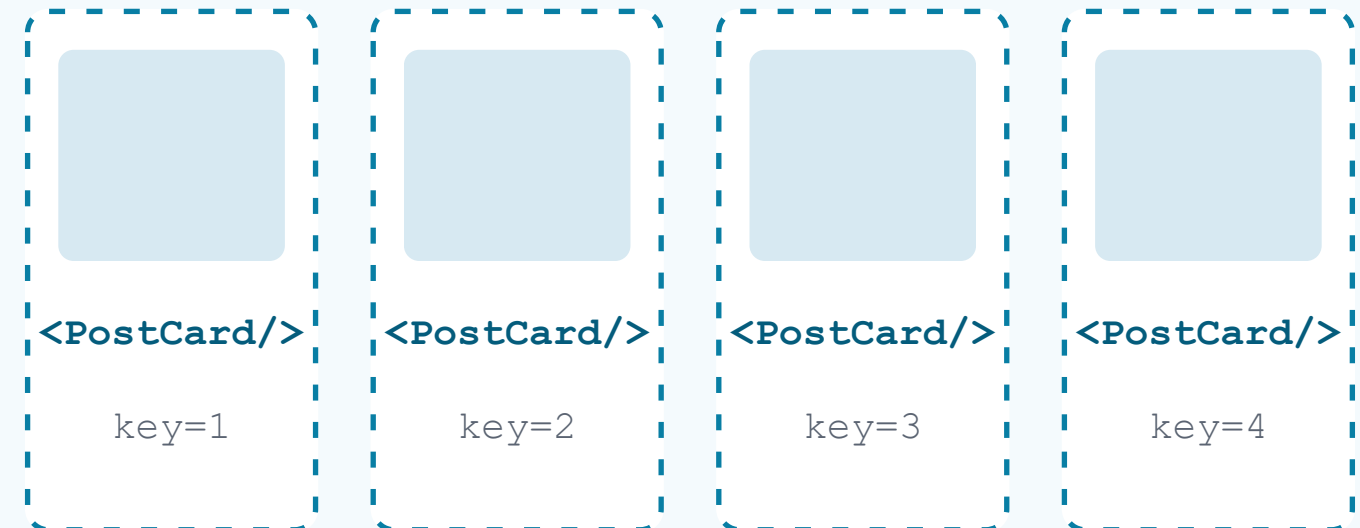
Every item needs a key

A unique `key={...}` tells React which item is which, so it can update lists efficiently.

REAL WORLD: Your Instagram feed, Amazon search results, Spotify playlists — all one array, one `.map()`, one reusable component.

```
posts = [ , , ,  ]
```

```
.map(post => <PostCard />)
```



4 items in → 4 components out. 1000 items? Same one line of code.

Lists with .map() in Code

The pattern behind every feed you scroll

```
CourseList.jsx

const courses = [
  { id: 1, name: 'Web Engineering' },
  { id: 2, name: 'Databases' },
  { id: 3, name: 'AI Basics' },
];

function CourseList() {
  return (
    <ul>
      {courses.map(c => (
        <li key={c.id}>{c.name}</li>
      ))}
    </ul>
  );
}
```

.map(...)

Transforms each object in the array into one element.

key={c.id}

Must be unique & stable. Avoid using the array index if items can move.

{ } braces

Again: curly braces let JS (the map call) live inside JSX.

REAL WORLD: Change the array → the list on screen updates itself.

Everything Together: A Tiny Todo App

State + events + a controlled form + a rendered list — in 20 lines

```
TodoApp.jsx
function TodoApp() {
  const [text, setText] = useState(''); // form state
  const [todos, setTodos] = useState([]); // list state

  function addTodo(e) {
    e.preventDefault();
    setTodos([...todos, text]); // add to list
    setText(''); // clear the box
  }

  return (
    <div>
      <form onSubmit={addTodo}>
        <input value={text}
          onChange={e => setText(e.target.value)} />
      </form>
      {todos.length === 0 && <p>Nothing yet!</p>}
      <ul>{todos.map((t, i) => <li key={i}>{t}</li>)}</ul>
    </div> ); }
```

Spot today's five concepts:

- ✓ **Component**
the function itself
- ✓ **State**
useState ×2
- ✓ **Event**
onSubmit / onChange
- ✓ **Controlled form**
value={text}
- ✓ **Conditional + list**
&& and .map()



You Can Now Think in React



Components — reusable UI bricks — functions returning JSX



useState — a component's memory; change it → UI updates



Events — onClick & friends run your functions



Controlled forms — value + onChange keep state in charge



Lists & conditions — .map() with keys · ternary · &&



Let's see if you can 'Think' in React

Homework: Build a tiny "To Do App" yourself.

Then add a delete button per item.

Practice Task - Counter Application

Create a React application that:

- Displays a counter initialized to 0.
- Has three buttons:
 - Increment (+1)
 - Decrement (-1)
 - Reset (sets the counter to 0)
- Display the counter value in the center of the page.
- If the value is positive, show the text "Positive".
- If the value is negative, show "Negative".
- Otherwise show "Zero".

Practice Task - Counter Application

```
function App() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <h1>Counter Application</h1>  
  
      <h2>{count}</h2>  
  
      <button onClick={() => setCount(count + 1)}>Increment</button>  
      <button onClick={() => setCount(count - 1)}>Decrement</button>  
      <button onClick={() => setCount(0)}>Reset</button>  
  
      {count > 0 ? ( <p>Positive</p> )  
        : count < 0 ? ( <p>Negative</p> )  
        : ( <p>Zero</p> )  
    }  
    </div>  
  );  
}
```

Practice Task - Student List Manager

Create a React application where users can:

- Enter a student's name.
- Click an **Add Student** button.
- Display all student names in a list.
- Show the total number of students.
- If there are no students, display "**No students added yet.**"

Bonus: Add a Delete button beside each student.

Practice Task - Student List Manager

```
function App() {  
  const [student, setStudent] = useState("");  
  const [students, setStudents] = useState([]);  
  
  const addStudent = () => {  
    if (student === "") return;  
  
    setStudents([...students, student]);  
    setStudent("");  
  };  
}
```

```
return (  
  <div>  
    <h1>Student List Manager</h1>  
  
    <input  
      type="text"  
      placeholder="Enter student name"  
      value={student}  
      onChange={(e) => setStudent(e.target.value)}  
    />  
  
    <button onClick={addStudent}>Add Student</button>  
  </div>  
)
```

Practice Task - Student List Manager

```
<h3>Total Students: {students.length}</h3>

{students.length === 0 && <p>No students added yet.</p>

{
  <ul>
    {students.map((name, index) => (
      <li key={index}> {name} </li>
    ))}
  </ul>
}
</div>
);
}
```

Thank You!